

项目总结

商城总结

这个项目是美容美体网的 PC 端商城，业务上覆盖商品展示和搜索、商品详情、购物车、下单支付，以及会员、积分、优惠券和订单查询。它有一个特点：商城交易和内容获客结合得比较紧，首页和商品详情页除了展示商品，也承载资讯、问答和 SEO 内容，把搜索流量引导到商品购买链路。

技术上，项目使用 ASP.NET Core MVC 和 Razor 渲染页面，前端通过 JavaScript 调用商品、会员、订单等后端 API。商城端主要负责页面展示和请求编排，订单创建、支付等核心业务由对应服务处理。商品详情页还涉及历史 URL 的 301 兼容、SEO 元信息，以及 MongoDB 详情数据快照，所以维护时需要特别关注页面可访问性、数据刷新和老链接兼容。

erp总结

项目与业务总结

这是公司内部使用的综合 ERP，核心是把营销获客、客户跟进、订单成交、发货售后、财务核算和经营分析串起来。销售、客服、运营、采购、财务和管理人员围绕同一批客户、商品和订单协作。

面试时记住一条业务链就够了：

渠道获客 → 线索分配 → 电话／微信跟进 → 下单 → 收款与发货 → 签收及售后 → 复购 → 经营分析

其中，客户模块管理资源归属、保护期和跟进记录；订单模块承接人工开单及平台订单，处理发货、物流、退换货和业绩归属；统计模块汇集各业务数据，分析业绩、转化、库存和投入产出。产品采购、财务支付、人事权限等模块为这条主线提供支撑。业务全景梳理 (line 49)

面试口述版

我参与的是一套公司内部 ERP，主要服务商品销售和客户经营。业务从广告、电商、直播等渠道获取线索开始，系统将线索分配给销售或客服，通过电话、微信和回访记录持续跟进；客户成交后，订单模块负责建单、收款确认、发货、物流、签收和售后，后续再通过会员权益、优惠券等方式促进复购。

这个项目的特点是业务状态比较复杂。比如订单“允许发货”、实际发货、签收、到款和业绩确认是不同概念；做报表时，也要区分订单金额、回款、签收业绩和利润的统计口径。系统按客户、订单、商品、统计等业务拆分服务，管理端主要使用 Vue 2 和 Element UI，后端主要是 .NET Framework 4.8 的 Web API，跨模块统计通过数据汇聚库完成。

如果面试官追问“你个人负责什么”，再接你实际做过一个模块，按“业务问题 → 你的改动 → 关键点 → 验证结果”讲；不要把整个 ERP 的功能说成自己独立完成。以上是根据仓库代码和文档做的静态梳理，不能据此确认每项功能目前都在线上启用。架构与数据访问规范 (line 1)

?"实现下单、微信扫码 / 支付宝支付、藏宝岛特殊订单分支、会员等级与积分兑换等核心交易逻辑。"这个可以怎么回答

这句话面试时建议按交易链路讲，并说清你负责的是商城端的流程编排。可以这样回答：

我负责 PC 商城交易流程的页面和接口对接。普通商品从购物车进入结算页后，页面会整理收货地址、商品、支付方式和优惠券等信息，调用订单服务创建订单，成功后跳转到支付页。藏宝岛商品有独立的订单类型，结算前要拦截它与普通商品混合下单，再走统一订单接口。支付页先获取订单的待付金额：微信侧展示扫码入口，支付宝侧调用支付服务获取跳转地址。会员业务还包括查询等级和积分、购买会员卡，以及按接口返回的兑换规则校验积分并提交兑换请求。订单创建、支付结果和积分扣减的最终处理由后端服务负责，我负责商城端的交互、参数组织、分支处理和失败提示。

这段话对应的代码依据包括：普通商品下单 (line 1217)、藏宝岛分支 (line 1075)、支付页 (line 196)、积分兑换 (line 1466)。

如果是写简历，原句可改为：

负责 PC 商城下单与支付流程的页面实现和接口对接，处理藏宝岛特殊订单分支，并实现会员等级展示、会员购买及积分兑换交互。

有两个细节要避免说过头：代码中的微信二维码指向站内支付页面，不能直接说自己实现了微信支付网关或支付回调；前端有提交防抖，但不能据此声称实现了订单服务端幂等。这两项若确实是你在其他服务中做的，再单独补充。

"主导前端组件体系与性能优化，落地长列表虚拟滚动、接口并发控制等，首屏加载时间降低约 30%-50%。"这个可以怎么回答

这句话面试时建议拆成“场景、做法、结果”来讲，并把性能指标说到具体页面：

我主要优化过 ERP 的统计报表页面。这类页面数据行数多、列也多，普通表格一次渲染大量 DOM，滚动和筛选时容易卡顿。我在相关表格中使用虚拟滚动，让页面主要渲染可视区域的行；对于 LTV 客户批量导出，则把分页请求按每批 10 个并发执行，控制请求峰值。优化时我会分别看接口耗时和浏览器渲染耗时，避免把后端查询慢误判成前端问题。组件层面，我会把实际负责的公共表格、筛选或分页组件作为例子，说明如何减少重复实现。

仓库能确认统计表格启用了虚拟滚动 (line 152)，以及 LTV 导出使用每批 10 个请求 (line 2080)。但我没找到“首屏加载降低 30%-50%”的测量记录；虚拟滚动改善的是大表格渲染，批量导出的并发控制也不能直接证明首屏提速。

如果你有当时的测试数据，可以补一句：“在相同页面、数据量 and 环境下，可交互时间从 X 秒降到 Y 秒，重复测试后约降低 Z%。”如果没有，简历建议改成：“优化统计报表大数据量渲染与批量导出性能，落地虚拟滚动和分批并发请求控制。”“主导组件体系”也最好准备好自己负责的具体组件和推广范围，再使用“主导”这个词。